Práctica 2. Estrategia Blue/Green y Canary con Elastic Load Balancing

Pau López Núñez (paulopnun@alu.edu.gva.es)

9 de noviembre de 2026

IES Juan de Garay (València). Curso de Especialización en Recursos y Servicios en la Nube.

Resumen

Se parte de una web sencilla con dos versiones: la versión blue (fondo azul) y la versión green (fondo verde). Cada una está en su propia instancia EC2 y un balanceador de carga decide cuál recibe el tráfico. Con un balanceador de carga de red (NLB) se hace el cambio Blue/Green: primero solo responde blue y después se pasa a green sin cortar el servicio, manteniendo blue encendida para poder volver atrás. Como ampliación, con un balanceador de aplicaciones (ALB) se hace un despliegue Canary repartiendo el tráfico por pesos.

Todo se ha creado con la AWS CLI mediante scripts y también con un programa en Python (Boto3) que lo automatiza de principio a fin. Las capturas de la consola de AWS muestran los recursos creados.

Los recursos siguen la convención de nombres tres letras del servicio + iniciales: sec-pln-web, tar-pln-nlb, nlb-pln, alb-pln...

Entorno: instancias blue y green

El script 01-crear-entorno.sh crea un grupo de seguridad con SSH (22) y HTTP (80) y lanza dos instancias t2.micro con Amazon Linux 2023 y el par de claves vockey, una en cada zona de disponibilidad. En los datos de usuario se instala Apache y se genera una página con el nombre de la versión, el color de fondo y el hostname del servidor:

```
#!/bin/bash
# Servidor web con una página que muestra el color de la versión y el hostname.
# COLOR y NOMBRE se sustituyen al lanzar cada instancia.
dnf install -y httpd
systemctl enable --now httpd
cat > /var/www/html/index.html <<HTML
<!DOCTYPE html>
<html lang="es">
<head><meta charset="utf-8"><title>NOMBRE</title></head>
```

```
<body style="background:COLOR;color:white;font-family:sans-serif;text-align:center;padding-top:15%">
  <h1>Versión NOMBRE</h1>
  <h2>Servidor: $(hostname -f)</h2>
</body>
</html>
HTML
```

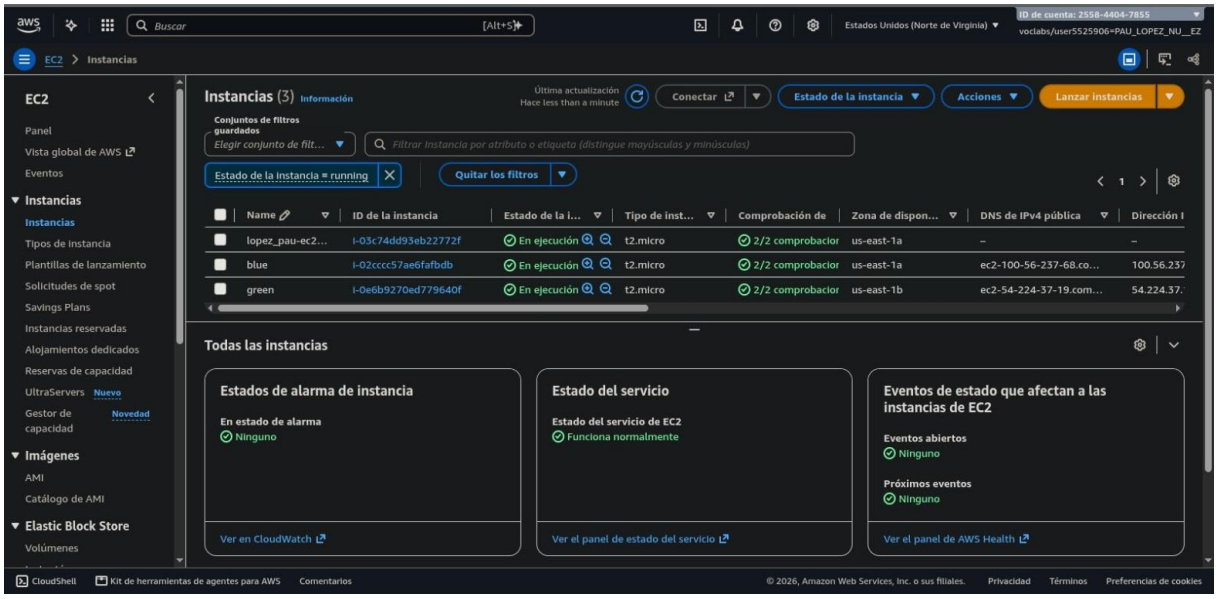
Comandos principales del script:

```
aws ec2 create-security-group --group-name sec-pln-web \
  --description "SSH y HTTP para los servidores blue y green" --vpc-id $VPC_ID
aws ec2 authorize-security-group-ingress --group-id $SG_EC2 --protocol tcp --port 22 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $SG_EC2 --protocol tcp --port 80 --cidr 0.0.0.0/0

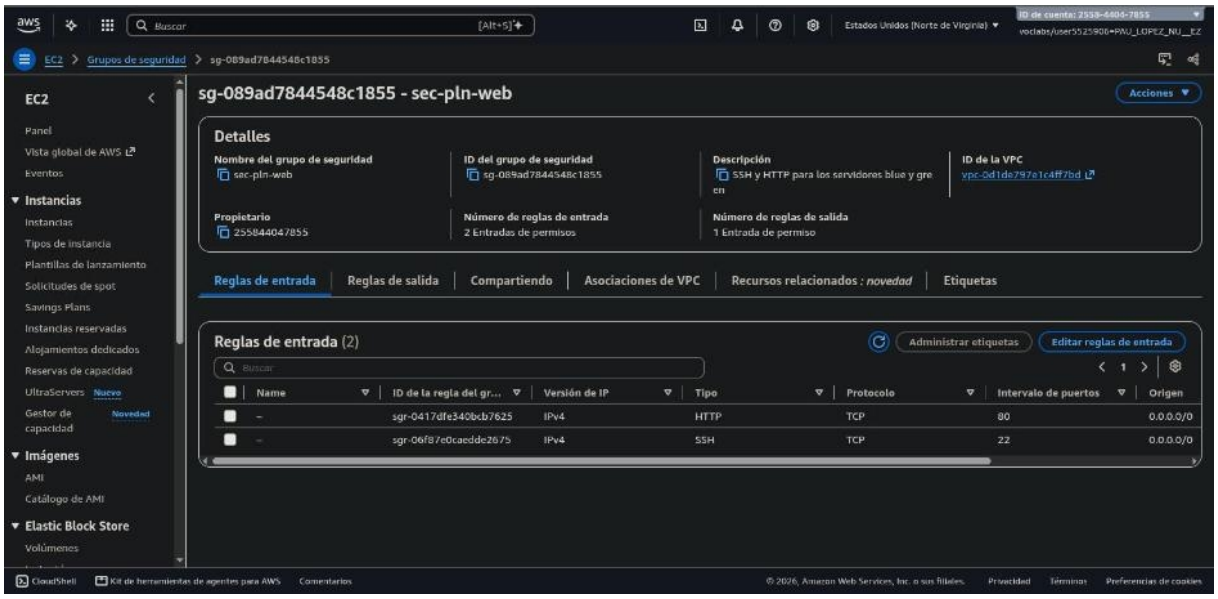
aws ec2 run-instances --image-id $AMI --instance-type t2.micro --key-name vockey \
  --subnet-id $SUBNET_A --security-group-ids $SG_EC2 --associate-public-ip-address \
  --user-data file:///tmp/ud-blue.sh \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=blue}]"
```

Salida de la ejecución:

```
Grupo de seguridad de las instancias: sg-089ad7844548c1855
blue (servidor1): i-02cccc57ae6fafbdb
green (servidor2): i-0e6b9270ed779640f
-----------------------------------------------------------------------------------------------------------
|                                          DescribeInstances                                              |
+-------+----------------------+----------+-------------+-----------------------------------------------+
|  blue |  i-02cccc57ae6fafbdb |  running |  us-east-1a |  ec2-100-56-237-68.compute-1.amazonaws.com    |
|  green|  i-0e6b9270ed779640f |  running |  us-east-1b |  ec2-54-224-37-19.compute-1.amazonaws.com     |
+-------+----------------------+----------+-------------+-----------------------------------------------+
```



Instancias blue (us-east-1a) y green (us-east-1b) en ejecución



Grupo de seguridad sec-pln-web con SSH y HTTP

Antes de poner el balanceador se comprobó con el DNS público de cada instancia que la página carga:

```
curl http://ec2-100-56-237-68.compute-1.amazonaws.com
curl http://ec2-54-224-37-19.compute-1.amazonaws.com

  <h1>Versión blue</h1>
  <h2>Servidor: ip-172-31-3-206.ec2.internal</h2>
  <h1>Versión green</h1>
  <h2>Servidor: ip-172-31-84-11.ec2.internal</h2>
```

Balanceador de carga de red con la CLI

El script 02-nlb-blue-green.sh crea el balanceador siguiendo los pasos del enunciado:

• Grupo de seguridad sec-pln-nlb con el puerto 80 de entrada.

• Grupo de destino tar-pln-nlb de tipo instancia, protocolo TCP y puerto 80, en la VPC por defecto y con las comprobaciones de estado por defecto. El tiempo de drenaje se baja a 10 segundos para que el cambio de versión sea rápido.

• Solo se registra la instancia blue.

• Balanceador nlb-pln de tipo network, expuesto a internet, en us-east-1a y us-east-1b, con balanceo entre zonas.

• Agente de escucha TCP 80 que reenvía al grupo de destino.

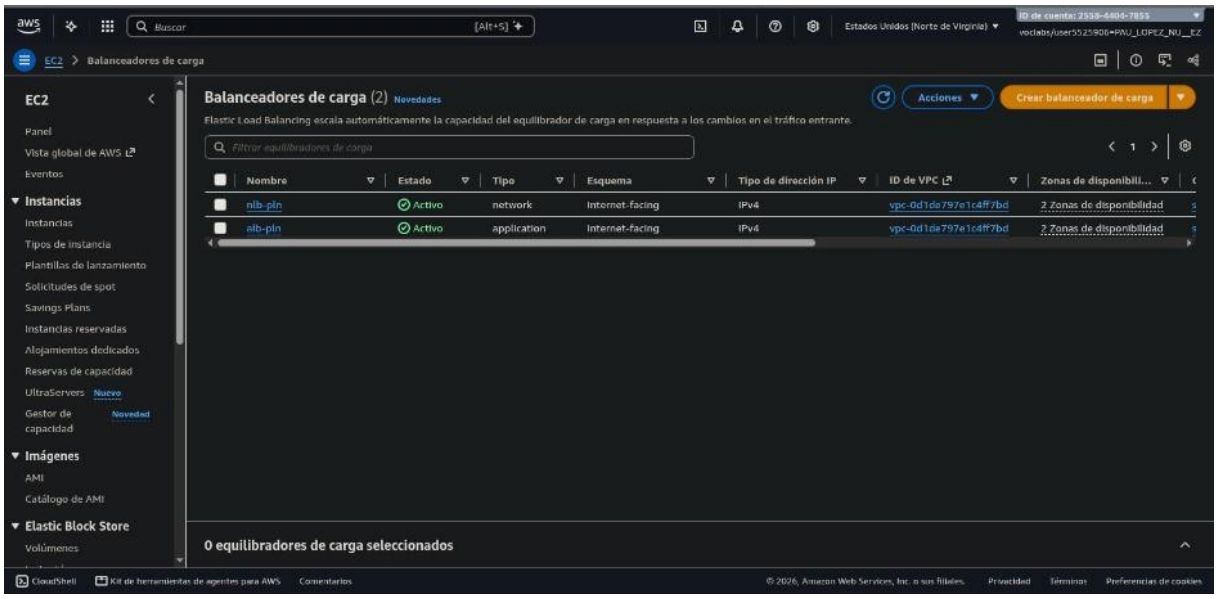
```
aws elbv2 create-target-group --name tar-pln-nlb --protocol TCP --port 80 \
  --vpc-id $VPC_ID --target-type instance
aws elbv2 register-targets --target-group-arn $TG_NLB --targets Id=$ID_BLUE,Port=80

aws elbv2 create-load-balancer --name nlb-pln --type network --scheme internet-facing \
  --subnets $SUBNET_A $SUBNET_B --security-groups $SG_NLB
```

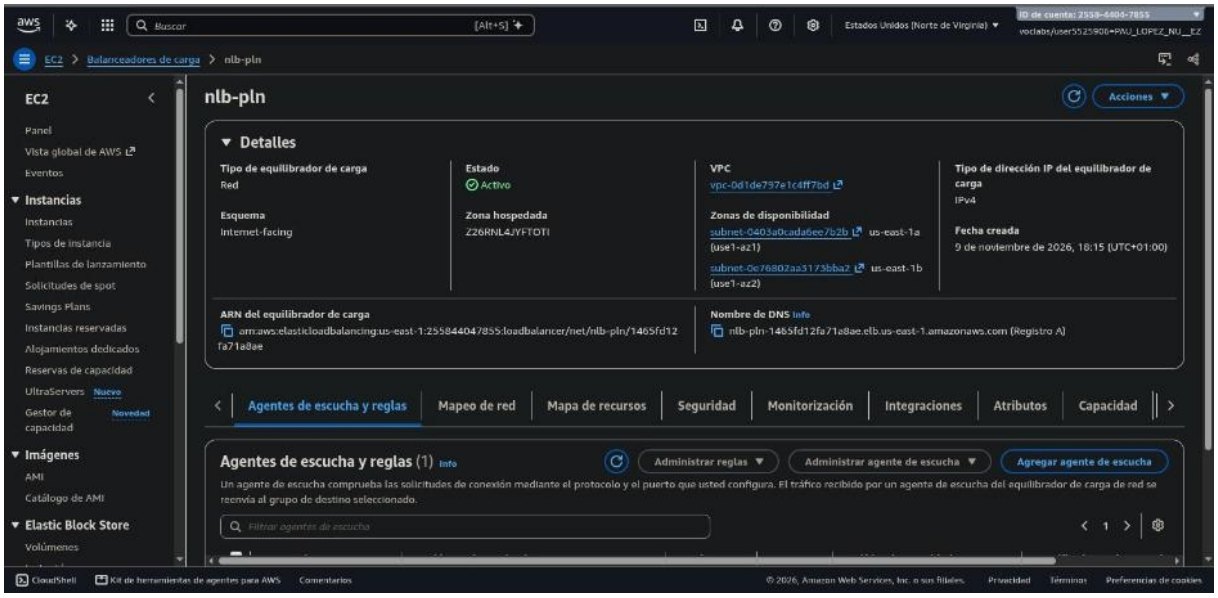
```
aws elbv2 create-listener --load-balancer-arn $NLB --protocol TCP --port 80 \
  --default-actions Type=forward,TargetGroupArn=$TG_NLB
```

Salida de ./02-nlb-blue-green.sh crear:

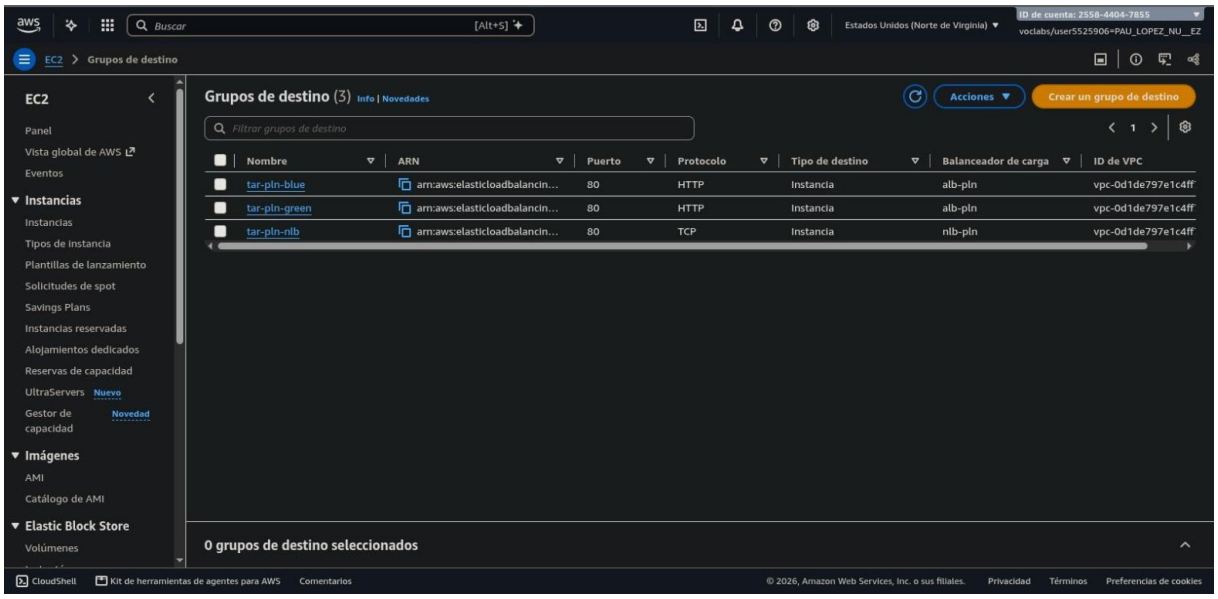
```
Esperando a que el NLB esté activo...
Esperando a que i-02cccc57ae6fafbdb esté healthy...
-------------------------------------
|       DescribeTargetHealth        |
+---------------------+-----------+
|  i-02cccc57ae6fafbdb |  healthy  |
+---------------------+-----------+
NLB: http://nlb-pln-1465fd12fa71a8ae.elb.us-east-1.amazonaws.com
```



Balanceadores nlb-pln (red) y alb-pln (aplicación) activos



Detalles del NLB: dos zonas de disponibilidad y nombre DNS

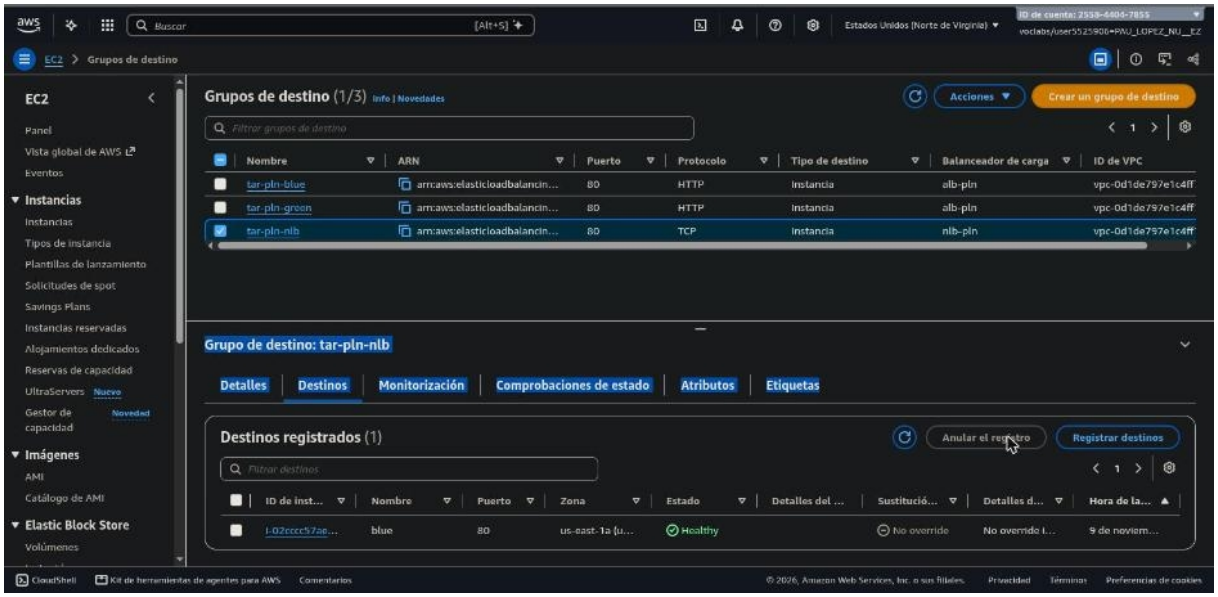


Grupos de destino: tar-pln-nlb (TCP) para el NLB y tar-pln-blue/green (HTTP) para el ALB

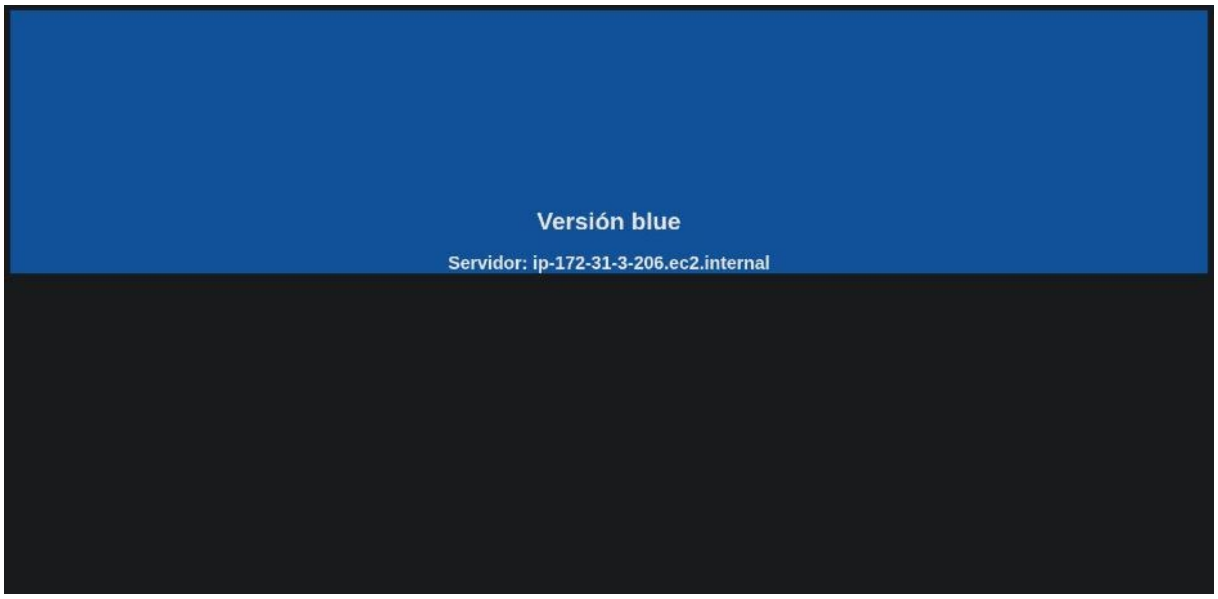
Primero la instancia blue

Con solo blue registrada en el grupo de destino, todas las peticiones al DNS del balanceador las responde blue. El script comprobar.sh hace varias peticiones abriendo una conexión nueva en cada una y cuenta qué versión contesta:

```
./comprobar.sh http://nlb-pln-1465fd12fa71a8ae.elb.us-east-1.amazonaws.com 20
     20 Versión blue
```



Grupo de destino del NLB con solo la instancia blue (healthy)



La web a través del NLB muestra la versión blue

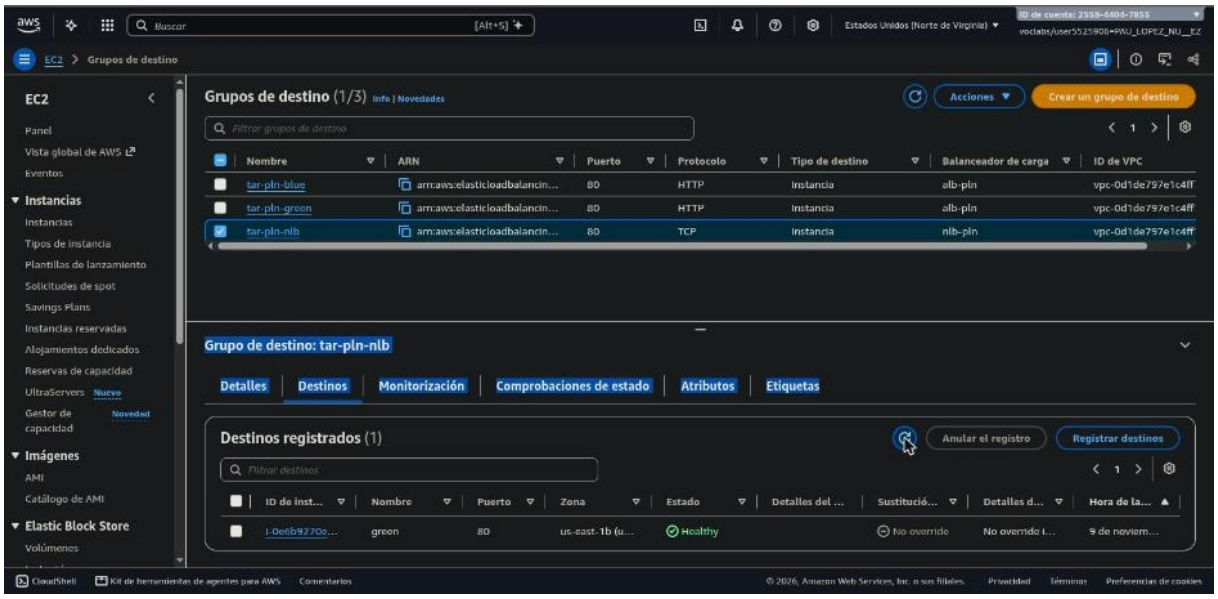
Después la instancia green

El cambio de versión se hace sin cortar el servicio: se registra green, se espera a que pase la comprobación de estado y solo entonces se retira blue. Blue no se apaga, así que si la versión nueva falla se puede volver atrás con ./02-nlb-blue-green.sh blue.

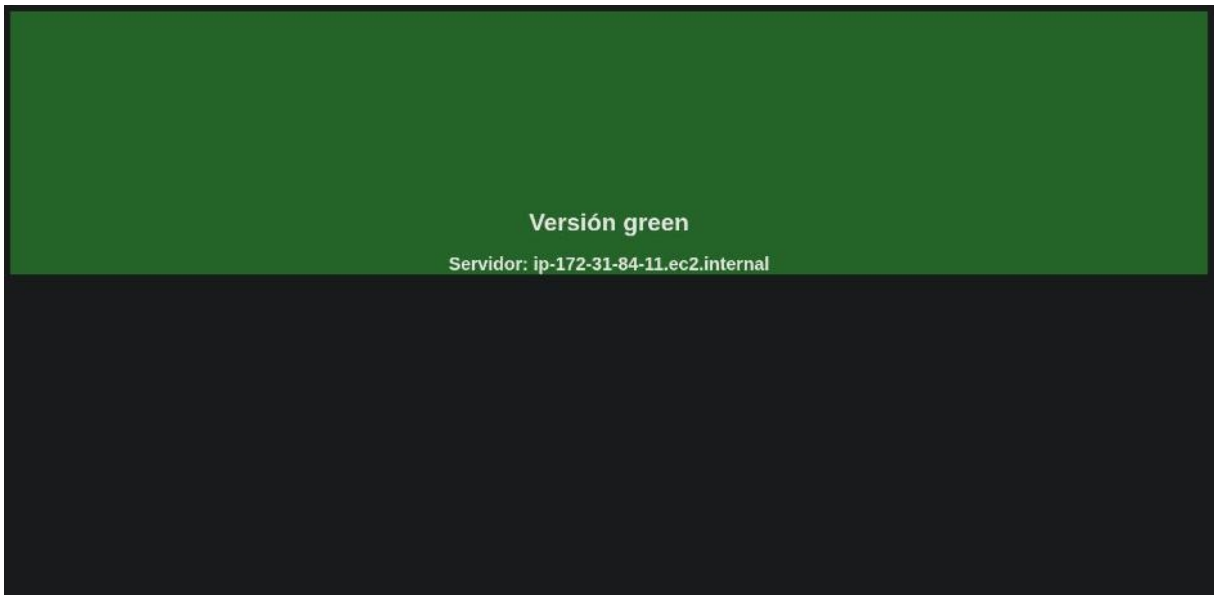
```
aws elbv2 register-targets --target-group-arn $TG_NLB --targets Id=$ID_GREEN,Port=80
aws elbv2 wait target-in-service --target-group-arn $TG_NLB --targets Id=$ID_GREEN,Port=80
aws elbv2 deregister-targets --target-group-arn $TG_NLB --targets Id=$ID_BLUE,Port=80
aws elbv2 wait target-deregistered --target-group-arn $TG_NLB --targets Id=$ID_BLUE,Port=80
```

Salida de ./02-nlb-blue-green.sh green y comprobación:

```
Esperando a que i-0e6b9270ed779640f esté healthy...
-------------------------------------
|       DescribeTargetHealth       |
+----------------------+-----------+
|  i-0e6b9270ed779640f |  healthy  |
+----------------------+-----------+
     20 Versión green
```



Grupo de destino del NLB tras el cambio: solo green

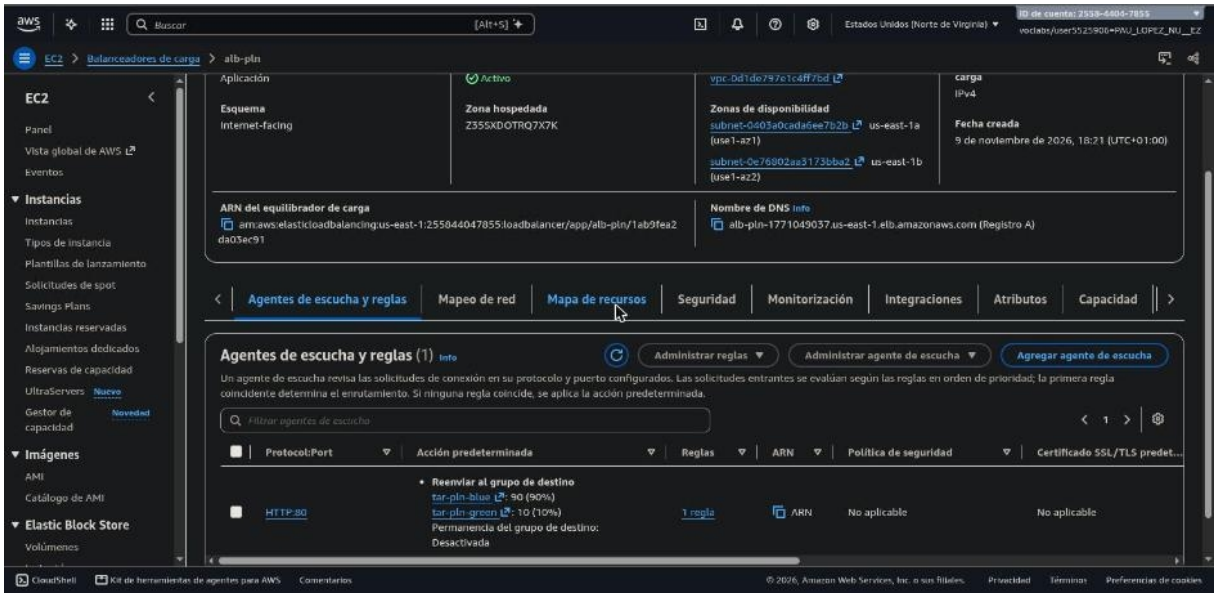


La misma URL del NLB muestra ahora la versión green

Ampliación: Canary con un balanceador de aplicaciones

El balanceador de red no permite repartir el tráfico por pesos entre grupos de destino, así que para la estrategia Canary se usa un ALB (alb-pln) con dos grupos de destino HTTP, tar-pln-blue y tar-pln-green. El agente de escucha reenvía a los dos con un peso cada uno y la versión nueva va recibiendo cada vez más tráfico:

```
aws elbv2 modify-listener --listener-arn $LISTENER_ALB --default-actions \
  "Type=forward,ForwardConfig={TargetGroups=[{TargetGroupArn=$TG_BLUE,Weight=90},
{TargetGroupArn=$TG_GREEN,Weight=10}]}"
```



Agente de escucha del ALB con 90 % a blue y 10 % a green

Reparto medido con comprobar.sh en cada paso:

Pesos blue / green	Respuestas blue	Respuestas green
100 / 0	20 de 20	0
90 / 10	36	4
50 / 50	25	15
0 / 100	0	40

Los pasos intermedios no son exactos porque son muestras pequeñas, pero se ve cómo la versión green va ganando tráfico hasta quedarse con todo.

Programa en Python con Boto3

El programa elb_blue_green.py hace lo mismo que los scripts de la CLI. Con la orden todo crea el entorno, el NLB con blue, hace el cambio a green, crea el ALB y recorre los pesos del Canary, comprobando el resultado en cada paso. También se puede usar por pasos (entorno, nlb, green, blue, canary, pesos, limpiar). Para probarlo sin tocar los recursos de la CLI se ejecutó con el sufijo pln2:

```
INI=pln2 python3 elb_blue_green.py todo

== 1. Entorno ==
Grupo de seguridad sec-pln2-web: sg-0876ef801d39904e0
blue: i-0559eb5711a699182
green: i-0fb6efaf37f741bef
  blue   i-0559eb5711a699182  us-east-1a  http://ec2-13-222-140-14.compute-1.amazonaws.com
  green  i-0fb6efaf37f741bef  us-east-1b  http://ec2-52-87-160-49.compute-1.amazonaws.com

== 2. NLB con la versión blue ==
Grupo de seguridad sec-pln2-nlb: sg-01809253180ef2ab6
Esperando a que el NLB esté activo...
Esperando a que i-0559eb5711a699182 esté healthy...
  i-0559eb5711a699182  healthy
NLB: http://nlb-pln2-9e6c201c60aa7d01.elb.us-east-1.amazonaws.com
  blue: 20

== 3. Blue/Green: paso a la versión green ==
Esperando a que i-0fb6efaf37f741bef esté healthy...
Versión activa en el NLB: green
  i-0fb6efaf37f741bef  healthy
  green: 20

== 4. Canary con ALB ==
Grupo de seguridad sec-pln2-alb: sg-029c5ad4b1c6afa38
Esperando a que el ALB esté activo...
Esperando a que i-0559eb5711a699182 esté healthy...
Esperando a que i-0fb6efaf37f741bef esté healthy...
Pesos: blue 100 / green 0
ALB: http://alb-pln2-1841900196.us-east-1.elb.amazonaws.com
  blue: 20
Pesos: blue 90 / green 10
  blue: 38, green: 2
Pesos: blue 50 / green 50
  blue: 19, green: 21
Pesos: blue 0 / green 100
  green: 40

INI=pln2 python3 elb_blue_green.py limpiar
Recursos eliminados
```

Limpieza

Al terminar, ./04-limpiar.sh borra los balanceadores, los grupos de destino, las instancias y los grupos de seguridad para no consumir crédito del laboratorio.

Repositorio y bibliografía

• Scripts, programa y evidencias: https://github.com/PauLopNun/curso-servicios-nube/tree/main/Despliegue-de-Servicios-Administrados-en-la-Nube/Practica-2-ELB-Blue-Green-Canary

• Elastic Load Balancing: https://docs.aws.amazon.com/es_es/elasticloadbalancing/latest/application/introduction.html

• Network Load Balancer: https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html

• Reparto por pesos en el ALB:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html